

# Thermo Fisher Interview Preparation

## Detailed Questions & Interview-Ready Answers

**Source:** Questions and scenarios are based on the uploaded interview transcript. The answers are recommended study answers, not claims that these exact answers were given in the interview.

**How to use:** Memorize the technical flow and key terms. Do not memorize every sentence. Keep answers direct, then expand only when the interviewer asks for details.

## 1. Introduction & Experience

### How many years of experience do you have?

**Answer:** I have around 4.7 years of total IT experience, with approximately 3.5 to 3.6 years of relevant DevOps experience. My recent work focuses on AWS, EKS, Kubernetes, Jenkins, Terraform, Docker and monitoring.

### How many years of relevant DevOps experience do you have?

**Answer:** I have around 3.5 to 3.6 years of relevant DevOps experience, mainly across AWS, EKS, CI/CD, Terraform, Docker, Kubernetes, Helm, ArgoCD and observability.

### What were you doing prior to DevOps?

**Answer:** I started my career as a CAE Engineer in the automotive domain. Later I moved toward cloud and DevOps, where I built my foundation in Linux, shell scripting, Git, CI/CD and AWS.

### Have you worked on on-premises as well as on cloud, or just AWS services?

**Answer:** My primary hands-on cloud experience is AWS. I have not independently operated a production on-premises Kubernetes environment.

### Can you tell me how is your infrastructure set up?

**Answer:** Our application is a containerized microservices-based web application running on EKS across multiple Availability Zones. Jenkins handles CI/CD, Terraform manages AWS infrastructure, ECR stores images, RDS is used for databases, S3 for object storage, Prometheus/Grafana for monitoring, ELK for logging and ArgoCD for GitOps-based deployment.

### According to your applications, what is your application?

**Answer:** It is a telecom-domain web application consisting of 50-plus microservices. The services are containerized and deployed on an EKS cluster spanning three Availability Zones.

## 2. EKS / Kubernetes Operations

### Have you done this setup or someone else has done this setup?

**Answer:** I joined after the initial EKS environment was created, so I did not independently build the production cluster from scratch. My work has mainly been maintenance, deployments, configuration changes and operations. I understand the end-to-end setup using EKS, VPC networking, node groups, IAM, VPC CNI, CoreDNS, kube-proxy and CSI drivers.

### Do you take care of the cluster upgrades?

**Answer:** I have not personally executed a production EKS upgrade. I understand the process: check version compatibility, back up critical resources, upgrade the control plane, update node groups and add-ons, validate workloads and monitor the cluster after the upgrade.

### What about the cluster backups?

**Answer:** We use Velero for Kubernetes backup and restore. The backup strategy should protect Kubernetes resources and, where required, persistent-volume data, with backups stored in object storage such as S3 and restoration tested periodically.

### How are you using Velero?

**Answer:** Velero is used for Kubernetes backup and disaster recovery. It can back up selected Kubernetes resources and, depending on configuration, persistent-volume data. Backup metadata is stored in object storage and resources can later be

restored into the cluster.

### **What kind of backups are you taking?**

**Answer:** Primarily Kubernetes resource backups such as namespaces, deployments, services, ConfigMaps and other selected resources. Stateful workloads can also have persistent-volume data protected depending on the configured snapshot or data-mover mechanism.

### **How are you scheduling the Velero backups?**

**Answer:** Velero supports scheduled backups. For example, I can configure a daily schedule for selected namespaces, define retention, monitor backup status and periodically test restore procedures.

### **What add-ons are you using in your Kubernetes cluster?**

**Answer:** For EKS, the core add-ons are AWS VPC CNI, CoreDNS and kube-proxy. We also use the Amazon EBS CSI driver for persistent block storage. Other components such as ingress, metrics and monitoring are additional platform components rather than the core EKS add-ons.

## **3. Storage — GP3 / NFS / PV / PVC / hostPath**

### **What StorageClass are you using in your Kubernetes cluster?**

**Answer:** For AWS block storage, we use a GP3-based StorageClass backed by the Amazon EBS CSI driver. A PVC requests storage from the StorageClass and the CSI driver provisions or attaches the corresponding EBS volume.

### **What add-on do you need to have GP3 StorageClass?**

**Answer:** We need the Amazon EBS CSI driver. It allows Kubernetes to dynamically provision and attach EBS volumes for PVCs, with the StorageClass specifying parameters such as the GP3 volume type.

### **How can I use this NFS storage as a StorageClass, create a PV, create a PVC and attach it to my pod?**

**Answer:** For dynamic provisioning, I would deploy a supported NFS CSI driver or NFS provisioner and configure it with the NFS server and export path. The StorageClass uses that provisioner. A PVC requests storage, the provisioner creates or binds a PV, and the pod mounts the PVC. For static provisioning, I can manually define an NFS PV and bind a PVC to it.

### **Why do you need a ConfigMap here for the NFS storage?**

**Answer:** A ConfigMap is not inherently required for NFS storage. The NFS server address and export path are storage configuration and can be supplied through the NFS CSI driver/provisioner or PV configuration. A Secret is only needed if the particular storage system requires authentication.

### **How can NFS storage be used here?**

**Answer:** NFS provides shared network file storage that multiple pods can mount, subject to the NFS server and access-mode configuration. Kubernetes accesses it through an NFS CSI driver/provisioner or a statically defined NFS PV.

### **Can you tell me how this works and how can I use this NFS storage as a StorageClass?**

**Answer:** The StorageClass points to an NFS provisioner or CSI driver. The provisioner knows the NFS server and export location and handles provisioning. A PVC requests storage, Kubernetes creates or binds a PV, and the pod mounts the PVC.

### **If I want to use the host path storage, how would you use it?**

**Answer:** I can define a hostPath volume pointing to a directory on the Kubernetes node and mount it into the pod. However, hostPath is node-local; if the pod moves to another node, the same data is not automatically available. Therefore it is generally not preferred for production shared persistent storage.

## **4. Kubernetes Troubleshooting & Probes**

### **What could be the possible reasons for CrashLoopBackOff?**

**Answer:** I first check the pod status and events using `kubectl describe pod`, then inspect current and previous container logs. Common causes include application startup failure, wrong command or arguments, missing ConfigMaps or Secrets, failed dependencies, bad environment variables, liveness-probe failure, OOMKilled and application errors.

## **What is liveness and readiness probe?**

**Answer:** A liveness probe determines whether the container is healthy enough to continue running; repeated failure can cause Kubernetes to restart it. A readiness probe determines whether the pod is ready to receive traffic; failure removes the pod from Service endpoints without necessarily restarting it.

## **A container takes 5, 10 or 15 minutes to start and probes cause CrashLoopBackOff. How do you fix it?**

**Answer:** I would use a startupProbe for a slow-starting application. It gives the application time to initialize before liveness and readiness checks take effect. I would also tune the probe thresholds based on measured startup time rather than simply disabling health checks.

## **What happens if we remove the liveness and readiness probe?**

**Answer:** Without liveness, Kubernetes loses the automated mechanism to restart a stuck or unhealthy container. Without readiness, an unhealthy pod may continue receiving traffic. Removing both can hide failures, so it is better to configure the probes correctly.

## **What if we remove both probes?**

**Answer:** The pod can remain Running even when the application is unhealthy, and there is no readiness-based traffic protection or liveness-based restart. For slow startup, startupProbe is the better solution.

# **5. CNI / Cluster Architecture**

## **What CNI are you using in the EKS cluster?**

**Answer:** We use AWS VPC CNI. It assigns VPC IP addresses to pods and integrates pod networking with the AWS VPC. Pod IP capacity is therefore influenced by the networking and ENI/IP capacity of the worker nodes.

## **I want a six-node cluster with three control plane and three worker nodes, with one worker node on-premises. How do you set it up?**

**Answer:** First I would clarify the platform because a standard EKS managed-node architecture does not simply allow an arbitrary on-premises worker to be added as a normal managed node. For a true hybrid requirement, I would use a supported EKS hybrid architecture, establish secure network connectivity between on-premises and AWS, validate Kubernetes version, networking and node requirements, and then use labels and taints/tolerations for workload placement.

## **How do you set up this cluster?**

**Answer:** I would first define the supported hybrid architecture and networking, provision the AWS and on-premises components, establish secure connectivity, configure node registration and validate CNI, DNS, certificates and node health. After that, node labels, taints and tolerations can control workload placement.

# **6. Docker**

## **How about your experience in Docker?**

**Answer:** I have hands-on experience writing Dockerfiles and using Docker build in Jenkins pipelines. I containerize applications, scan images, tag them with an immutable identifier such as a Git commit SHA, push them to ECR and deploy them through Kubernetes and GitOps.

## **Suppose I have some UI code in a GitHub repo and ask you to create a Docker image. How would you do it?**

**Answer:** I would check out the repository, understand the UI build process, create a Dockerfile with an appropriate base image, use a multi-stage build where applicable, build the application, copy only the required runtime artifacts into the final image, test it and scan it before pushing.

## **After creating a Docker image, where/how would you push it?**

**Answer:** I would authenticate to the target registry, tag the image with the repository name and an immutable version such as the Git commit SHA, and push it. In our AWS environment, that would typically be Amazon ECR.

## **After pushing the image, how would you deploy it using Docker or Kubernetes/EKS?**

**Answer:** For Kubernetes, I would update the Helm values or GitOps repository with the new immutable image tag. ArgoCD detects the Git change and syncs it to EKS. I would then verify rollout status, pod health and application health.

## 7. Docker Build / UI Troubleshooting

**My UI changes are not showing after rebuilding and redeploying. How would you troubleshoot it?**

**Answer:** I would verify the source commit used by the build, confirm the Docker build context contains the changed file, check Docker cache, inspect the image contents, verify the pushed image tag or digest, confirm Kubernetes is running the new image and check browser/CDN caching if necessary.

**During Docker build, how do you check whether the latest file was copied into the image?**

**Answer:** I would inspect the Dockerfile and build context, use `--no-cache` when required, then run a temporary container from the built image and inspect the target path with a shell. I can also use `docker history` to inspect layers. This verifies whether the expected file is actually present in the image.

## 8. CI/CD Pipeline

**Tell me how do you create this pipeline?**

**Answer:** I would design the pipeline as checkout, quality and security validation, build and test, Docker image build, vulnerability scan, push to ECR, update the GitOps repository, deploy to Dev, run automated validation, then promote to Stage. Production would perform the required validations and pause at a manual approval gate before deployment.

**How do you configure those pre-checks in detail?**

**Answer:** I would implement each check as an explicit Jenkins stage or gate. First verify the Dev deployment result, then use `kubectl rollout status` and pod readiness checks, followed by an application-level health check such as `curl` against a health endpoint or a UI smoke test. If all checks pass, promotion continues; otherwise the pipeline stops. For Production, I would add Dev and Stage validation plus a Jenkins approval gate restricted to authorized stakeholders.

## 9. Closing

**You have any questions?**

**Answer:** Yes. I would ask about the team's current DevOps architecture, the main production challenges, the level of production ownership expected in this role, and what success would look like during the first three to six months.

## High-Priority Revision Areas

- Kubernetes Storage: PV, PVC, StorageClass, EBS CSI, NFS CSI/provisioner, hostPath, access modes and reclaim policy.
- Velero: backup, restore, scheduling, S3/object storage and persistent-volume protection.
- Troubleshooting: CrashLoopBackOff, OOMKilled, probes, events, logs and rollout debugging.
- Docker: build context, COPY, cache, `--no-cache`, image inspection and immutable tags.
- EKS architecture: VPC CNI, CoreDNS, kube-proxy, CSI, node groups and hybrid architecture.
- Jenkins gates: rollout status, pod readiness, application health checks and production approval.